

- Description

This module manages a smart well water level monitoring system, ensuring that:

1. **The water level in the well is continuously monitored to maintain stable water supply.**
2. **If the water level drops below the minimum threshold, the pump is activated to extract water.**
3. **If the water level rises above the maximum threshold, the pump is deactivated to prevent overflow.**
4. **A log of all water level readings, pump activations, and shutdowns is maintained for water resource management.**

This system helps optimize water usage, prevent shortages, and improve automation in water extraction.

- Specification

module SmartWellWaterControl

var currentWaterLevel, maxWaterLevel, minWaterLevel: real

var pumpActive: boolean

process updateWaterLevel(changeAmount: real)

functional scenario form:

currentWaterLevel >= minWaterLevel && pumpActive = pumpActive, Water level maintained.

||

currentWaterLevel < minWaterLevel && pumpActive = true, Water pump activated.

||

currentWaterLevel > maxWaterLevel && pumpActive = false, Water pump deactivated.

end process

end module

- Java Code

```
package code;
class SmartWell {
    private double maxWaterLevel;
    private double minWaterLevel;
    private double currentWaterLevel;
    private boolean pumpActive;
    public SmartWell(double maxWaterLevel, double minWaterLevel, double
initialWaterLevel) {
        this.maxWaterLevel = maxWaterLevel;
        this.minWaterLevel = minWaterLevel;
        this.currentWaterLevel = initialWaterLevel;
        this.pumpActive = false;
    }
}
```

```

    }
    public void updateWaterLevel(double changeAmount) {
        currentWaterLevel += changeAmount;
        managePumpControl();
        System.out.printf("Current Water Level: %.2f meters\n", current
WaterLevel);
        if (pumpActive) {
            System.out.println("Water Pump Activated.");
        } else {
            System.out.println("Water Pump Deactivated.");
        }
    }
}
private void managePumpControl() {
    if (currentWaterLevel < minWaterLevel) {
        pumpActive = true;
    } else if (currentWaterLevel > maxWaterLevel) {
        pumpActive = false;
    }
}
}
}

public class c63_SmartWellSystem {
    public static void main(String[] args) {
        SmartWell well = new SmartWell(10.0, 3.0, 6.0);
        // testcase-VT:
        well.updateWaterLevel(-4.0);
        well.updateWaterLevel(5.0);
        well.updateWaterLevel(4.0);
        well.updateWaterLevel(-2.0);

        // testcase-FT:
        // well.updateWaterLevel(0.0);
        // well.updateWaterLevel(-1.7976931348623157E308);
        // well.updateWaterLevel(4.9E-324);
        // well.updateWaterLevel(1.7976931348623157E308);

        // testcase-Z3:
        // well.updateWaterLevel(-4.0);
        // well.updateWaterLevel(5.0);
        // well.updateWaterLevel(4.0);
        // well.updateWaterLevel(-2.0);

        // testcase-UVT:
        well.updateWaterLevel(-4.0);
        well.updateWaterLevel(5.0);
    }
}

```

```

        well.updateWaterLevel(4.0);
        well.updateWaterLevel(-2.0);
    }
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c63_SmartWellSystem.java	100.0 %	6	0	6
> c63_SmartWellSystem		0	0	0
> SmartWell	100.0 %	6	0	6

3. Vibration testing instruction coverage: 96.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	86	20,663	20,749
▼ src	0.4 %	86	20,663	20,749
▼ code	0.4 %	86	20,663	20,749
▼ c63_SmartWellSystem.java	96.6 %	86	3	89
> c63_SmartWellSystem	87.0 %	20	3	23
> SmartWell	100.0 %	66	0	66

4. Fuzz testing branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.6 %	5	837	842
▼ src	0.6 %	5	837	842
▼ code	0.6 %	5	837	842
▼ c63_SmartWellSystem.java	83.3 %	5	1	6
> c63_SmartWellSystem		0	0	0
> SmartWell	83.3 %	5	1	6

5. Fuzz testing instruction coverage: 93.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	83	20,666	20,749
▼ src	0.4 %	83	20,666	20,749
▼ code	0.4 %	83	20,666	20,749
▼ c63_SmartWellSystem.java	93.3 %	83	6	89
> c63_SmartWellSystem	87.0 %	20	3	23
> SmartWell	95.5 %	63	3	66

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c63_SmartWellSystem.java	100.0 %	6	0	6
> c63_SmartWellSystem		0	0	0
> SmartWell	100.0 %	6	0	6

7. Z3 instruction coverage: 96.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	86	20,663	20,749
▼ src	0.4 %	86	20,663	20,749
▼ code	0.4 %	86	20,663	20,749
▼ c63_SmartWellSystem.java	96.6 %	86	3	89
> c63_SmartWellSystem	87.0 %	20	3	23
> SmartWell	100.0 %	66	0	66

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c63_SmartWellSystem.java	100.0 %	6	0	6
> c63_SmartWellSystem		0	0	0
> SmartWell	100.0 %	6	0	6

9. Unimproved V-method instruction coverage: 96.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	86	20,663	20,749
▼ src	0.4 %	86	20,663	20,749
▼ code	0.4 %	86	20,663	20,749
▼ c63_SmartWellSystem.java	96.6 %	86	3	89
> c63_SmartWellSystem	87.0 %	20	3	23
> SmartWell	100.0 %	66	0	66